Django Complete Notes + Django ORM Complete Notes

Beginner to Advanced — Full Study Guide

এই notes-এ Django-এর ৪০টি topic এবং Django ORM-এর ৩০টি topic অন্তর্ভুক্ত করার structure রয়েছে। এটি Python/Django backend development, REST API development, practical projects এবং interview preparation-এর জন্য তৈরি।

Part A:

Table of Contents

1. Introduction to Django
2. Python Prerequisites
3. Environment Setup
4. Project and App Structure
5. URL Routing
6. Views — Beginner to Advanced
7. Templates and Template Language
8. Static and Media Files
9. Models and Database
10. Django ORM — Beginner to Advanced
11. CRUD Operations
12. Forms and ModelForms
13. Django Admin
14. Authentication and Authorization
15. Sessions and Cookies
16. Middleware
17. Model Relationships
18. Query Optimization
19. Django REST Framework
20. Serializers and Validation
21. API Views, ViewSets and Routers
22. API Authentication and Permissions
23. Filtering, Searching, Ordering and Pagination
24. Signals, Custom Managers and Commands
25. Error Handling and Logging
26. Testing and Debugging
27. Django Security
28. Caching
29. Background Tasks with Celery
30. Email Integration
31. File Uploads and Storage
32. Environment Variables and Configuration
33. Git and GitHub Workflow
34. Production Deployment
35. Performance Optimization
36. Docker and CI/CD

37. Advanced Django Concepts
38. Practical Projects
39. Interview Questions
40. Final Checklist

---

1. Introduction to Django

Django হলো Python-এর একটি high-level web framework, যা secure, maintainable এবং database-driven web applications ও APIs তৈরি করতে সাহায্য করে।

Key Features

- Built-in authentication and authorization
- Django ORM
- URL routing
- Template engine
- Forms and validation
- Admin panel
- Security protections
- Automated testing support
- Database integration

Django MVT Architecture

- **Model:** Database structure ও data access।
- **View:** Request process করে response তৈরি করে।
- **Template:** HTML presentation তৈরি করে।

```
Client
  |
  v
URL Dispatcher
  |
  v
View
  |
  +------> Model ------> Database
  |
  +------> Template
  |
  v
HTTP Response
```

Install Django

```
python -m pip install django
python -m django --version
```

Create a project

```
django-admin startproject config .
python manage.py startapp core
python manage.py runserver
```

---

2. Python Prerequisites

Django শেখার আগে নিচের Python concepts জানা প্রয়োজন:

- Variables and data types
- Lists, tuples, sets and dictionaries
- Conditions and loops
- Functions and arguments
- Classes and objects
- Inheritance and polymorphism
- Exception handling
- Modules and packages
- Decorators
- Iterators and generators
- Context managers
- Type hints
- Virtual environments

Example

```
def calculate_total(price: float, quantity: int) -> float:
    return price * quantity


class Product:
    def __init__(self, name: str, price: float):
        self.name = name
        self.price = price

    def get_price(self) -> float:
        return self.price


product = Product("Keyboard", 1500)
print(product.get_price())
```

---

3. Environment Setup

Create a virtual environment

```
python -m venv .venv
```

Windows:

```
.venv\Scripts\activate
```

Linux/macOS:

```
source .venv/bin/activate
```

Install Django

```
python -m pip install django
python -m pip freeze > requirements.txt
```

Install dependencies later

```
python -m pip install -r requirements.txt
```

Initialize project

```
django-admin startproject config .
python manage.py startapp products
```

Add the app in `settings.py`:

```
INSTALLED_APPS = [
    # Default Django apps...
    "products",
]
```

Run migrations

```
python manage.py migrate
python manage.py createsuperuser
python manage.py runserver
```

---

4. Project and App Structure

```
myproject/
├── manage.py
├── config/
│   ├── settings.py
│   ├── urls.py
│   ├── asgi.py
│   └── wsgi.py
├── products/
│   ├── migrations/
│   ├── admin.py
│   ├── apps.py
│   ├── models.py
│   ├── tests.py
│   ├── views.py
│   └── urls.py
├── templates/
├── static/
├── media/
└── requirements.txt
```

Important files

- `manage.py` : Django management commands |
- `settings.py` : Project configuration |
- `urls.py` : URL routing |
- `models.py` : Database models |
- `views.py` : Request/response logic |
- `admin.py` : Admin configuration |
- `tests.py` : Automated tests |
- `migrations/` : Database schema changes |

---

5. URL Routing

`config/urls.py`:

```
from django.contrib import admin
from django.urls import include, path

urlpatterns = [
    path("admin/", admin.site.urls),
    path("products/", include("products.urls")),
]
```

`products/urls.py`:

```
from django.urls import path
from . import views

app_name = "products"

urlpatterns = [
    path("", views.product_list, name="list"),
    path("<int:pk>/", views.product_detail, name="detail"),
]
```

`products/views.py`:

```
from django.http import HttpResponse


def product_list(request):
    return HttpResponse("Product list")


def product_detail(request, pk):
    return HttpResponse(f"Product ID: {pk}")
```

URL converters

- `str`
- `int`
- `slug`
- `uuid`
- `path`

Reverse URL lookup

```
from django.urls import reverse

url = reverse("products:detail", kwargs={"pk": 10})
```

Template:

```
<a href="{% url 'products:list' %}">Products</a>
```

---

6. Views — Beginner to Advanced

Function-based view

```
from django.http import JsonResponse
from django.shortcuts import render


def home(request):
    return render(request, "home.html")


def health_check(request):
    return JsonResponse({"status": "ok"})
```

Handling POST requests

```
def contact(request):
    if request.method == "POST":
        name = request.POST.get("name", "")
        return JsonResponse({"name": name})

    return render(request, "contact.html")
```

Class-based view

```
from django.views import View
from django.http import HttpResponse


class HomeView(View):
    def get(self, request):
        return HttpResponse("GET request")

    def post(self, request):
        return HttpResponse("POST request")
```

URL:

```
path("", HomeView.as_view(), name="home")
```

Generic views

```
from django.views.generic import ListView, DetailView
from .models import Product


class ProductListView(ListView):
    model = Product
    template_name = "products/list.html"
    context_object_name = "products"
    paginate_by = 10


class ProductDetailView(DetailView):
    model = Product
    template_name = "products/detail.html"
```

---

7. Templates and Template Language

`templates/home.html`:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>{{ title }}</title>
</head>
<body>
    <h1>{{ title }}</h1>

    {% if products %}
        <ul>
            {% for product in products %}
                <li>{{ product.name }}</li>
            {% endfor %}
        </ul>
    {% else %}
        <p>No products available.</p>
    {% endif %}
</body>
</html>
```

View:

```
from django.shortcuts import render


def home(request):
    context = {
        "title": "Product Store",
        "products": [],
    }
    return render(request, "home.html", context)
```

Important template syntax

- `{{ variable }}`: Output a variable |
- `{% if %}`: Conditional logic |
- `{% for %}`: Loop |
- `{% url %}`: Reverse URL |
- `{% extends %}`: Template inheritance |
- `{% block %}`: Define a template block |
- `{% include %}`: Include a reusable template |

Template inheritance

`base.html`:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <title>{% block title %}My Site{% endblock %}</title>
</head>
<body>
    <nav>My Website</nav>
```

```
    <main>
        {% block content %}{% endblock %}
    </main>
</body>
</html>
```

`home.html`:

```
{% extends "base.html" %}

{% block title %}Home{% endblock %}

{% block content %}
<h1>Welcome</h1>
{% endblock %}
```

---

8. Static and Media Files

Static configuration

```
STATIC_URL = "static/"
STATIC_ROOT = BASE_DIR / "staticfiles"
```

Template:

```
{% load static %}

<link rel="stylesheet" href="{% static 'css/style.css' %}">
```

Collect production static files:

```
python manage.py collectstatic
```

Media configuration

```
MEDIA_URL = "media/"
MEDIA_ROOT = BASE_DIR / "media"
```

Example model:

```
from django.db import models


class Product(models.Model):
    name = models.CharField(max_length=150)
    image = models.ImageField(
        upload_to="products/",
        blank=True,
    )
```

Install Pillow if required:

```
python -m pip install Pillow
```

Production-এ uploaded media-এর জন্য suitable storage ও access controls ব্যবহার করুন।

---

9. Models and Database

```
from django.db import models


class Product(models.Model):
    name = models.CharField(max_length=150)
    description = models.TextField(blank=True)
    price = models.DecimalField(max_digits=10, decimal_places=2)
    stock = models.PositiveIntegerField(default=0)
    is_active = models.BooleanField(default=True)
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)

    class Meta:
        ordering = ["-created_at"]

    def __str__(self):
        return self.name
```

Common field types

- `CharField`
- `TextField`
- `IntegerField`
- `PositiveIntegerField`
- `DecimalField`
- `BooleanField`
- `DateField`
- `DateTimeField`
- `EmailField`
- `URLField`
- `UUIDField`
- `JSONField`
- `FileField`
- `ImageField`

Important field options

- `null=True` : Database NULL অনুমোদন।
- `blank=True` : Validation-এ blank অনুমোদন।
- `default=` : Default value।
- `unique=True` : Unique constraint।
- `db_index=True` : Index।
- `choices=` : Allowed choices।
- `related_name=` : Reverse relation name।

---

10. Django ORM — Beginner to Advanced

ORM Python objects দিয়ে database operations করতে সাহায্য করে।

```
from products.models import Product

Product.objects.all()
Product.objects.filter(is_active=True)
Product.objects.get(pk=1)
Product.objects.exclude(stock=0)
```

Create

```
Product.objects.create(
    name="Keyboard",
    price="1500.00",
    stock=20,
)
```

Update

```
Product.objects.filter(pk=1).update(stock=25)
```

Delete

```
Product.objects.filter(pk=1).delete()
```

QuerySet laziness

```
qs = Product.objects.filter(is_active=True)
qs = qs.order_by("name")

products = list(qs)
```

QuerySet সাধারণত evaluation-এর আগে database query execute করে না। Iteration, `list()`, `len()` বা অন্যান্য evaluation operation query trigger করতে পারে।

---

11. CRUD Operations

CRUD means Create, Read, Update and Delete.

Create

```
product = Product.objects.create(
    name="Mouse",
    price="800.00",
    stock=50,
)
```

Read

```
Product.objects.all()
Product.objects.get(pk=1)
Product.objects.filter(is_active=True)
```

Update

```
product = Product.objects.get(pk=1)
product.price = "950.00"
product.save(update_fields=["price"])
```

Delete

```
product = Product.objects.get(pk=1)
product.delete()
```

Safe object lookup

```
from django.shortcuts import get_object_or_404

product = get_object_or_404(Product, pk=1)
```

---

12. Forms and ModelForms

`forms.py`:

```
from django import forms
from .models import Product


class ContactForm(forms.Form):
    name = forms.CharField(max_length=100)
    email = forms.EmailField()
    message = forms.CharField(widget=forms.Textarea)


class ProductForm(forms.ModelForm):
    class Meta:
        model = Product
        fields = ["name", "description", "price", "stock"]

    def clean_price(self):
        price = self.cleaned_data["price"]

        if price < 0:
            raise forms.ValidationError(
                "Price cannot be negative."
            )

        return price
```

Form processing

```
from django.shortcuts import redirect, render


def product_create(request):
    if request.method == "POST":
        form = ProductForm(request.POST)

        if form.is_valid():
            form.save()
            return redirect("products:list")
    else:
        form = ProductForm()
```

```
    return render(
        request,
        "products/form.html",
        {"form": form},
    )
```

Important methods:

- `is_valid()`
- `cleaned_data`
- `clean_<field>()`
- `clean()`
- `errors`
- `save()`

---

13. Django Admin

`admin.py`:

```
from django.contrib import admin
from .models import Product


@admin.register(Product)
class ProductAdmin(admin.ModelAdmin):
    list_display = (
        "id",
        "name",
        "price",
        "stock",
        "is_active",
    )
    list_filter = ("is_active",)
    search_fields = ("name", "description")
    list_per_page = 25
    ordering = ("name",)
```

Create an administrator:

```
python manage.py createsuperuser
```

Admin features:

- `list_display`
- `list_filter`
- `search_fields`
- `readonly_fields`
- `list_editable`
- `fieldsets`
- Custom admin actions

---

14. Authentication and Authorization

Authentication identifies the user. Authorization determines what the user can do.

Login-required view

```
from django.contrib.auth.decorators import login_required
from django.shortcuts import render


@login_required
def dashboard(request):
    return render(request, "dashboard.html")
```

Configuration

```
LOGIN_URL = "login"
LOGIN_REDIRECT_URL = "dashboard"
LOGOUT_REDIRECT_URL = "login"
```

Built-in authentication views

```
from django.contrib.auth import views as auth_views
from django.urls import path

urlpatterns = [
    path(
        "login/",
        auth_views.LoginView.as_view(
            template_name="registration/login.html"
        ),
        name="login",
    ),
    path(
        "logout/",
        auth_views.LogoutView.as_view(),
        name="logout",
    ),
]
```

Use a POST form with CSRF protection for logout.

Permission checking

```
from django.contrib.auth.decorators import permission_required


@permission_required(
    "products.change_product",
    raise_exception=True,
)
def edit_product(request):
    ...
```

Custom user models should generally be considered at the beginning of the project, before initial migrations.

---

15. Sessions and Cookies

Sessions

```
from django.http import JsonResponse


def cart_example(request):
    request.session["cart_count"] = 3

    count = request.session.get("cart_count", 0)

    return JsonResponse({"cart_count": count})
```

Cookies

```
from django.http import HttpResponse


def set_preference(request):
    response = HttpResponse("Preference saved")

    response.set_cookie(
        "theme",
        "dark",
        max_age=86400,
        secure=True,
        httponly=True,
        samesite="Lax",
    )

    return response
```

Read a cookie:

```
theme = request.COOKIES.get("theme", "light")
```

Sensitive information plain-text cookies-এ রাখবেন না। Secure cookie settings production-এর HTTPS configuration-এর সঙ্গে মিলিয়ে ব্যবহার করুন।

---

16. Middleware

Middleware request ও response processing-এর মাঝখানে কাজ করে।

Custom middleware

```
import logging
import time

logger = logging.getLogger(__name__)


class RequestTimingMiddleware:
    def __init__(self, get_response):
        self.get_response = get_response

    def __call__(self, request):
        start = time.perf_counter()

        response = self.get_response(request)
```

```
        duration = time.perf_counter() - start

        logger.info(
            "path=%s duration=%.4f status=%s",
            request.path,
            duration,
            response.status_code,
        )

        return response
```

Add to `MIDDLEWARE`:

```
  "core.middleware.RequestTimingMiddleware",
```

Uses:

- Request logging
- Security headers
- Request IDs
- Custom request processing
- Response modification

Middleware order গুরুত্বপূর্ণ।

---

17. Model Relationships

One-to-One

```
  from django.conf import settings
  from django.db import models


  class Profile(models.Model):
      user = models.OneToOneField(
          settings.AUTH_USER_MODEL,
          on_delete=models.CASCADE,
          related_name="profile",
      )
      bio = models.TextField(blank=True)
```

One-to-Many

```
  class Category(models.Model):
      name = models.CharField(max_length=100)


  class Product(models.Model):
      category = models.ForeignKey(
          Category,
          on_delete=models.PROTECT,
          related_name="products",
      )
      name = models.CharField(max_length=150)
```

Many-to-Many

```
class Student(models.Model):
    name = models.CharField(max_length=100)


class Course(models.Model):
    title = models.CharField(max_length=100)
    students = models.ManyToManyField(
        Student,
        related_name="courses",
    )
```

`on_delete` options

- `CASCADE`
- `PROTECT`
- `SET_NULL`
- `SET_DEFAULT`
- `DO_NOTHING`

---

18. Query Optimization

N+1 problem

Potentially inefficient:

```
products = Product.objects.all()

for product in products:
    print(product.category.name)
```

Optimized:

```
products = Product.objects.select_related("category")

for product in products:
    print(product.category.name)
```

Common techniques

- `select_related()`
- `prefetch_related()`
- `only()`
- `defer()`
- `values()`
- `values_list()`
- `exists()`
- `count()`
- `iterator()`
- `bulk_create()`
- `bulk_update()`
- Database indexes
- `QuerySet.explain()`

Measure performance before optimizing.

---

19. Django REST Framework

Install:

```
python -m pip install djangorestframework
```

Add to `INSTALLED_APPS`:

```
"rest_framework",
```

Serializer

`serializers.py`:

```
from rest_framework import serializers
from .models import Product


class ProductSerializer(serializers.ModelSerializer):
    class Meta:
        model = Product
        fields = ["id", "name", "price", "stock"]
```

API view

```
from rest_framework.decorators import api_view
from rest_framework.response import Response
from .models import Product
from .serializers import ProductSerializer


@api_view(["GET"])
def product_list_api(request):
    products = Product.objects.all()

    serializer = ProductSerializer(
        products,
        many=True,
    )

    return Response(serializer.data)
```

HTTP methods

- GET: Read
- POST: Create
- PUT: Replace
- PATCH: Partial update
- DELETE: Delete

---

20. Serializers and Validation

```
from rest_framework import serializers
from .models import Product
```

```
class ProductSerializer(serializers.ModelSerializer):
    class Meta:
        model = Product
        fields = ["id", "name", "price", "stock"]

    def validate_name(self, value):
        if not value.strip():
            raise serializers.ValidationError(
                "Name cannot be empty."
            )
        return value.strip()

    def validate_price(self, value):
        if value < 0:
            raise serializers.ValidationError(
                "Price cannot be negative."
            )
        return value
```

Validate incoming data

```
serializer = ProductSerializer(data=request.data)

if serializer.is_valid():
    product = serializer.save()
else:
    print(serializer.errors)
```

Important methods:

- `validate_<field>()`
- `validate()`
- `create()`
- `update()`
- `is_valid()`
- `save()`

Use explicit fields and configure read-only/write-only fields appropriately.

---

21. API Views, ViewSets and Routers

APIView

```
from rest_framework.views import APIView
from rest_framework.response import Response
from rest_framework import status
from .models import Product
from .serializers import ProductSerializer


class ProductListCreateAPIView(APIView):
    def get(self, request):
        products = Product.objects.all()
```

```
        serializer = ProductSerializer(products, many=True)
        return Response(serializer.data)

    def post(self, request):
        serializer = ProductSerializer(data=request.data)
        serializer.is_valid(raise_exception=True)
        serializer.save()

        return Response(
            serializer.data,
            status=status.HTTP_201_CREATED,
        )
```

ModelViewSet

```
from rest_framework.viewsets import ModelViewSet


class ProductViewSet(ModelViewSet):
    queryset = Product.objects.all()
    serializer_class = ProductSerializer
```

Router

```
from rest_framework.routers import DefaultRouter

router = DefaultRouter()
router.register("products", ProductViewSet, basename="product")

urlpatterns = router.urls
```

---

22. API Authentication and Permissions

Authenticated access

```
from rest_framework.permissions import IsAuthenticated
from rest_framework.viewsets import ModelViewSet


class ProductViewSet(ModelViewSet):
    queryset = Product.objects.all()
    serializer_class = ProductSerializer
    permission_classes = [IsAuthenticated]
```

Object-level permission

```
from rest_framework.permissions import BasePermission, SAFE_METHODS


class IsOwnerOrReadOnly(BasePermission):
    def has_object_permission(self, request, view, obj):
        if request.method in SAFE_METHODS:
            return True

        return obj.owner_id == request.user.id
```

The model must have an appropriate `owner` field.
Authentication options:

- Session authentication
- Basic authentication
- Token authentication
- JWT through a suitable package
- OAuth 2.0 through a suitable library

Authentication and authorization are different responsibilities.

---

23. Filtering, Searching, Ordering and Pagination

Install:

```
python -m pip install django-filter
```

Add:

```
"django_filters",
```

ViewSet:

```
from rest_framework.filters import SearchFilter, OrderingFilter
from django_filters.rest_framework import DjangoFilterBackend
from rest_framework.viewsets import ModelViewSet


class ProductViewSet(ModelViewSet):
    queryset = Product.objects.all()
    serializer_class = ProductSerializer

    filter_backends = [
        DjangoFilterBackend,
        SearchFilter,
        OrderingFilter,
    ]

    filterset_fields = ["is_active", "category"]
    search_fields = ["name", "description"]
    ordering_fields = ["name", "price", "created_at"]
    ordering = ["-id"]
```

Query examples

```
/api/products/?is_active=true
/api/products/?search=keyboard
/api/products/?ordering=-price
/api/products/?category=2
```

Pagination

```
REST_FRAMEWORK = {
    "DEFAULT_PAGINATION_CLASS": (
        "rest_framework.pagination.PageNumberPagination"
    ),
```

```
    "PAGE_SIZE": 20,
}
```

Pagination types:

- `PageNumberPagination`
- `LimitOffsetPagination`
- `CursorPagination`

---

24. Signals, Custom Managers and Commands

Signal

```
from django.db.models.signals import post_save
from django.dispatch import receiver


@receiver(post_save, sender=Product)
def product_saved(sender, instance, created, **kwargs):
    if created:
        print(f"Created product: {instance.name}")
```

Register signal receivers reliably through the app configuration.

Custom manager

```
class ActiveProductManager(models.Manager):
    def get_queryset(self):
        return super().get_queryset().filter(is_active=True)
```

Custom management command

Create:
`products/management/commands/list_products.py`

```
from django.core.management.base import BaseCommand
from products.models import Product


class Command(BaseCommand):
    help = "List all products"

    def handle(self, *args, **options):
        for product in Product.objects.all():
            self.stdout.write(product.name)
```

Run:

```
python manage.py list_products
```

---

25. Error Handling and Logging

Safe lookup

```
from django.shortcuts import get_object_or_404

product = get_object_or_404(Product, pk=1)
```

Logging

```
import logging

logger = logging.getLogger(__name__)


def process_order(order_id):
    logger.info("Processing order_id=%s", order_id)

    try:
        # Business logic
        pass
    except Exception:
        logger.exception(
            "Failed to process order_id=%s",
            order_id,
        )
        raise
```

Never log passwords, authentication tokens or other sensitive information.

---

26. Testing and Debugging

Model test

```
from django.test import TestCase
from products.models import Product


class ProductModelTests(TestCase):
    def test_product_string_representation(self):
        product = Product.objects.create(
            name="Keyboard",
            price="1500.00",
            stock=10,
        )

        self.assertEqual(str(product), "Keyboard")
```

Run tests:

```
python manage.py test
```

View test

```
from django.test import TestCase
from django.urls import reverse


class ProductViewTests(TestCase):
    def test_product_list_status(self):
        response = self.client.get(reverse("products:list"))
        self.assertEqual(response.status_code, 200)
```

DRF test

```
from rest_framework.test import APITestCase
from rest_framework import status


class ProductAPITests(APITestCase):
    def test_api_response(self):
        response = self.client.get("/products/api/")

        self.assertEqual(
            response.status_code,
            status.HTTP_200_OK,
        )
```

Test model behavior, permissions, validation, API responses and failure cases.

---

27. Django Security

Risk	Protection
SQL injection	ORM or parameterized SQL
XSS	Template autoescaping
CSRF	CSRF middleware and token
Clickjacking	X-Frame-Options protection
Weak passwords	Password validators
Unauthorized access	Authentication and permissions
Secret leakage	Environment

	variables or secret manager
Unsafe uploads	File validation and safe storage
Insecure deployment	HTTPS and secure settings

Production settings

```
DEBUG = False

ALLOWED_HOSTS = ["example.com"]

SESSION_COOKIE_SECURE = True
CSRF_COOKIE_SECURE = True
```

Check deployment configuration:

```
python manage.py check --deploy
```

Also configure HTTPS, trusted origins, secrets, database credentials, secure cookies and monitoring.

---

28. Caching

Cache API

```
from django.core.cache import cache


def get_dashboard_data():
    data = cache.get("dashboard_data")

    if data is None:
        data = {"total_products": 100}
        cache.set("dashboard_data", data, timeout=300)

    return data
```

The example data is illustrative.

Invalidate cache

```
cache.delete("dashboard_data")
```

Caching options:

- Per-site cache
- Per-view cache

- Template fragment cache
- Low-level cache API
- Redis or Memcached

Cache keys must account for user or tenant context when results differ by authorization.

---

29. Background Tasks with Celery

Install:

```
python -m pip install celery redis
```

`config/celery.py`:

```
import os
from celery import Celery

os.environ.setdefault("DJANGO_SETTINGS_MODULE", "config.settings")

app = Celery("config")
app.config_from_object("django.conf:settings", namespace="CELERY")
app.autodiscover_tasks()
```

`config/__init__.py`:

```
from .celery import app as celery_app

__all__ = ("celery_app",)
```

Settings:

```
CELERY_BROKER_URL = "redis://localhost:6379/0"
```

`products/tasks.py`:

```
from celery import shared_task


@shared_task
def generate_report():
    return "Report generated"
```

Run worker:

```
celery -A config worker --loglevel=INFO
```

Call task:

```
generate_report.delay()
```

Use Celery for background emails, reports, scheduled jobs and long-running tasks.

---

30. Email Integration

SMTP configuration

```
EMAIL_BACKEND = "django.core.mail.backends.smtp.EmailBackend"
EMAIL_HOST = "smtp.example.com"
EMAIL_PORT = 587
```

```
EMAIL_USE_TLS = True
EMAIL_HOST_USER = "your-account@example.com"
EMAIL_HOST_PASSWORD = "load-from-environment"
DEFAULT_FROM_EMAIL = "noreply@example.com"
```

Send email

```
from django.core.mail import send_mail


def send_welcome_email(user_email):
    send_mail(
        subject="Welcome",
        message="Thanks for registering.",
        from_email=None,
        recipient_list=[user_email],
        fail_silently=False,
    )
```

Keep credentials outside source code. Use the console email backend during development when appropriate.

---

31. File Uploads and Storage

Upload form

```
from django import forms


class UploadForm(forms.Form):
    file = forms.FileField()
```

Upload view

```
from django.shortcuts import render


def upload_file(request):
    if request.method == "POST":
        form = UploadForm(request.POST, request.FILES)

        if form.is_valid():
            uploaded_file = form.cleaned_data["file"]
            return render(request, "upload_success.html")
    else:
        form = UploadForm()

    return render(request, "upload.html", {"form": form})
```

Template

```
<form method="post" enctype="multipart/form-data">
    {% csrf_token %}
    {{ form.as_p }}
```

```
    <button type="submit">Upload</button>
</form>
```

Validate file size and content. Store untrusted uploads safely and configure private file access controls.

---

32. Environment Variables and Configuration

```
import os

SECRET_KEY = os.environ["DJANGO_SECRET_KEY"]

DEBUG = (
    os.environ.get("DJANGO_DEBUG", "False").lower() == "true"
)
```

Example environment values:

```
DJANGO_SECRET_KEY=replace-with-a-secret
DJANGO_DEBUG=False
DATABASE_URL=postgresql://user:password@localhost:5432/appdb
```

Never commit secrets or `.env` files to Git.
Use environment-specific settings, secret management, appropriate logging and database configuration.

---

33. Git and GitHub Workflow

Initial setup

```
git init
git add .
git commit -m "Initial Django project"
git branch -M main
git remote add origin YOUR_REPOSITORY_URL
git push -u origin main
```

Feature branch

```
git checkout -b feature/product-api

git add .
git commit -m "Add product API"

git push -u origin feature/product-api
```

`.gitignore`

```
.venv/
__pycache__/
*.py[cod]
.env
db.sqlite3
media/
staticfiles/
.coverage
.pytest_cache/
```

Use pull requests, code review, automated tests and meaningful commit messages.

---

34. Production Deployment

Typical architecture

```
Client
  |
 HTTPS
  |
Reverse Proxy / Load Balancer
  |
Gunicorn or another production server
  |
Django
  ├── PostgreSQL
  ├── Redis / Cache
  ├── Object Storage
  └── Celery Workers
```

Deployment checklist

- `DEBUG=False`
- Correct `ALLOWED_HOSTS`
- Environment-based secrets
- Production database
- Database migrations
- Static file collection
- HTTPS
- Production WSGI/ASGI server
- Logging and monitoring
- Database and media backups
- Health checks
- Rollback strategy

`runserver` is a development server, not a production server.

---

35. Performance Optimization

Database optimization

- Find slow queries.
- Reduce N+1 queries.
- Add justified indexes.
- Use pagination.
- Use bulk operations when appropriate.
- Avoid loading unnecessary fields.
- Inspect execution plans.

Application optimization

- Cache expensive repeated calculations.
- Move long-running tasks to background workers.
- Profile CPU and memory.

- Avoid unnecessary middleware and signal side effects.
- Monitor latency and error rates.

Measure first, then optimize.

---

36. Docker and CI/CD

Dockerfile

```
FROM python:3.12-slim

ENV PYTHONDONTWRITEBYTECODE=1
ENV PYTHONUNBUFFERED=1

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

CMD ["gunicorn", "config.wsgi:application", "--bind", "0.0.0.0:8000"]
```

Ensure Gunicorn and all required dependencies are installed.

Typical services

- Django web service
- PostgreSQL
- Redis
- Celery worker

CI/CD pipeline

```
Push Code
    |
Install Dependencies
    |
Lint and Run Tests
    |
Build Docker Image
    |
Deploy to Staging
    |
Smoke Tests
    |
Production Deployment
```

Use versioned images, secure secrets, migration plans and rollback procedures.

---

37. Advanced Django Concepts

Custom QuerySet

```
class ProductQuerySet(models.QuerySet):
    def active(self):
        return self.filter(is_active=True)
```

```
    def in_stock(self):
        return self.filter(stock__gt=0)
```

Transactions

```
from django.db import transaction


@transaction.atomic
def create_order():
    # Create order and order items.
    pass
```

Async view

```
from django.http import JsonResponse


async def async_health(request):
    return JsonResponse({"status": "ok"})
```

Async views do not automatically make every database operation asynchronous. Verify compatibility for the Django version and libraries you use.

Additional advanced topics

- Custom middleware
- Database routers
- Multi-tenancy
- Service-layer architecture
- Background jobs
- Database constraints
- Async request handling
- Query optimization
- Domain-driven design principles

---

38. Practical Projects

Project 1: Blog Application

Features:

- User authentication
- Posts, categories and comments
- CRUD operations
- Search and pagination
- Admin management

Project 2: E-commerce Backend

Features:

- Products and categories
- Shopping cart
- Orders and order items
- Authentication and permissions

- PostgreSQL
- REST API
- Payment integration

Project 3: School Management System

Features:

- Students and teachers
- Courses and enrollments
- Attendance
- Role-based permissions
- Reporting

Project 4: Production-Ready REST API

Features:

- Django REST Framework
- Authentication
- Filtering and pagination
- Automated tests
- PostgreSQL
- Docker
- CI/CD
- Deployment and monitoring

Recommended capstone

Inventory and Order Management API:

- User
- Category
- Product
- Warehouse
- Inventory
- Order
- OrderItem
- Payment

Implement transaction-safe order creation, stock validation, permissions, query optimization, testing and deployment.

---

39. Django Interview Questions

Q1. What is Django?

Django is a high-level Python web framework.

Q2. What is MVT?

Model, View and Template.

Q3. What is the difference between a project and an app?

A project contains configuration and applications; an app provides a specific area of functionality.

Q4. What is the difference between `makemigrations` and `migrate`?

`makemigrations` creates migration files. `migrate` applies migrations to the database.

Q5. What is the difference between `get()` and `filter()`?

`get()` expects exactly one result. `filter()` returns a QuerySet.

Q6. What is authentication?

Verifying the identity of a user or client.

Q7. What is authorization?

Determining which actions a user or client can perform.

Q8. What is middleware?

A component that processes requests and responses around the view.

Q9. What is Django REST Framework?

A toolkit for building Web APIs with Django.

Q10. How do you secure a Django application?

Use HTTPS, safe settings, proper authentication and permissions, secure secret management, validation, CSRF protection and deployment checks.

Q11. What is Celery?

A distributed task queue used for background and scheduled tasks.

Q12. Why use database indexes?

To improve selected query patterns, with trade-offs in storage and write performance.

---

40. Final Checklist

Fundamentals

- Python OOP, exceptions and decorators
- Virtual environments
- Project and app structure
- URL routing
- FBV and CBV
- Templates

Database

- Models and field types
- Migrations
- CRUD operations
- QuerySets and filtering
- Relationships
- Transactions
- Indexes

APIs

- Forms and validation
- Serializers
- APIViews and ViewSets
- Routers
- Authentication and permissions
- Filtering and pagination
- Error handling

Production

- Automated tests

- Logging
- Security configuration
- Environment variables
- Caching
- Celery
- PostgreSQL
- Docker
- CI/CD
- Deployment

Portfolio

- Complete CRUD project
- REST API project
- README and API documentation
- Automated tests
- GitHub repository
- Deployed application

Part B:

Table of Contents

1. Database Basics
2. Django ORM
3. Project Setup
4. Django Models and Fields
5. Migrations
6. CRUD Operations
7. QuerySets and Filtering
8. Field Lookups and Q Objects
9. Model Relationships
10. Ordering and Limiting Results
11. `select_related()` and `prefetch_related()`
12. Aggregation and Annotation
13. F Expressions
14. Conditional Expressions
15. Subquery and Exists
16. Managers and Custom QuerySets
17. Database Transactions
18. Raw SQL
19. Query Optimization and Indexes
20. Bulk Operations
21. Multiple Databases
22. Fixtures
23. Django Signals
24. Abstract Models and Proxy Models
25. Validation, Constraints and Security

26. Python and Django Introspection
27. Interview Questions
28. Quick ORM Cheat Sheet
29. Learning Roadmap
30. Capstone Project

---

1. Database Basics

Important concepts:

- Database
- Table
- Row
- Column
- Primary key
- Foreign key
- Index
- Constraint
- Transaction

SQL:

```
SELECT name, price
FROM products_product
WHERE price >= 1000
ORDER BY price DESC;
```

Django ORM:

```
Product.objects.filter(
    price__gte=1000
).order_by("-price").values("name", "price")
```

Understanding SQL helps you debug and optimize ORM queries.

---

2. Django ORM

```
from products.models import Product

Product.objects.all()
Product.objects.filter(is_active=True)
Product.objects.get(pk=1)
Product.objects.exclude(stock=0)
```

QuerySet evaluation

```
qs = Product.objects.filter(is_active=True)

# Evaluate the QuerySet
products = list(qs)
```

Other evaluation operations include iteration, `len()` and some terminal methods.

Manager

```
Product.objects.all()
```

`objects` is the default model manager.

---

3. Project Setup

```
python -m venv .venv
python -m pip install django

django-admin startproject config .
python manage.py startapp products

python manage.py makemigrations
python manage.py migrate
python manage.py shell
```

Shell example:

```
from products.models import Product

Product.objects.all()
```

---

4. Django Models and Fields

```
from django.db import models


class Product(models.Model):
    name = models.CharField(max_length=150)
    price = models.DecimalField(max_digits=10, decimal_places=2)
    stock = models.PositiveIntegerField(default=0)
    is_active = models.BooleanField(default=True)
    created_at = models.DateTimeField(auto_now_add=True)

    class Meta:
        ordering = ["-created_at"]
        indexes = [
            models.Index(fields=["is_active", "created_at"]),
        ]

    def __str__(self):
        return self.name
```

Field options

- `null=True`
- `blank=True`
- `default=`
- `unique=True`
- `db_index=True`
- `choices=`
- `editable=False`
- `related_name=`

Choices

```
class Order(models.Model):
    class Status(models.TextChoices):
        PENDING = "pending", "Pending"
        PAID = "paid", "Paid"
        CANCELLED = "cancelled", "Cancelled"

    status = models.CharField(
        max_length=20,
        choices=Status.choices,
        default=Status.PENDING,
    )
```

---

5. Migrations

```
python manage.py makemigrations
python manage.py makemigrations products
python manage.py migrate
python manage.py showmigrations
python manage.py sqlmigrate products 0001
```

Workflow:

1. Update model definitions.
2. Generate migration files.
3. Review generated migrations.
4. Apply migrations.
5. Commit migration files to Git.

Review destructive migrations and data migrations carefully.

---

6. CRUD Operations

Create

```
Product.objects.create(
    name="Keyboard",
    price="1500.00",
    stock=20,
)
```

Read

```
Product.objects.all()
Product.objects.get(pk=1)
Product.objects.filter(stock__gt=0)
```

Update

```
product = Product.objects.get(pk=1)
product.price = "1600.00"
product.save()
```

Bulk update

```
Product.objects.filter(is_active=False).update(stock=0)
```

Delete

```
Product.objects.filter(stock=0).delete()
```

Bulk operations do not behave exactly like calling each instance's `save()` or `delete()` method.

---

7. QuerySets and Filtering

```
Product.objects.all()

Product.objects.filter(price__gte=1000)

Product.objects.exclude(is_active=False)

Product.objects.filter(
    is_active=True,
    stock__gt=0,
)
```

Chaining

```
products = (
    Product.objects
    .filter(is_active=True)
    .exclude(stock=0)
    .order_by("price")
)
```

Single object

```
product = Product.objects.get(pk=1)
```

Possible exceptions:

- `Product.DoesNotExist`
- `Product.MultipleObjectsReturned`

Safe view lookup

```
from django.shortcuts import get_object_or_404

product = get_object_or_404(Product, pk=1)
```

---

8. Field Lookups and Q Objects

Common lookups

```
Product.objects.filter(name__exact="Mouse")
Product.objects.filter(name__iexact="mouse")
Product.objects.filter(name__contains="key")
Product.objects.filter(name__icontains="KEY")
Product.objects.filter(price__gt=1000)
Product.objects.filter(price__gte=1000)
Product.objects.filter(price__lt=5000)
Product.objects.filter(price__lte=5000)
```

```
Product.objects.filter(id__in=[1, 2, 3])
Product.objects.filter(price__range=(500, 2000))
Product.objects.filter(category__isnull=True)
Product.objects.filter(name__startswith="D")
Product.objects.filter(name__endswith="r")
```

Q objects

```
from django.db.models import Q

products = Product.objects.filter(
    Q(name__icontains="keyboard") |
    Q(description__icontains="keyboard")
)
```

AND:

```
Product.objects.filter(
    Q(is_active=True) & Q(stock__gt=0)
)
```

NOT:

```
Product.objects.filter(~Q(is_active=False))
```

---

9. Model Relationships

ForeignKey

```
class Category(models.Model):
    name = models.CharField(max_length=100)


class Product(models.Model):
    category = models.ForeignKey(
        Category,
        on_delete=models.PROTECT,
        related_name="products",
    )
    name = models.CharField(max_length=150)
```

Access:

```
product.category
category.products.all()
```

OneToOneField

```
from django.conf import settings


class Profile(models.Model):
    user = models.OneToOneField(
        settings.AUTH_USER_MODEL,
        on_delete=models.CASCADE,
        related_name="profile",
    )
```

ManyToManyField

```
class Tag(models.Model):
    name = models.CharField(max_length=50)


class Article(models.Model):
    title = models.CharField(max_length=200)
    tags = models.ManyToManyField(
        Tag,
        related_name="articles",
    )
```

Access:

```
article.tags.all()
tag.articles.all()
```

Relationship filtering

```
Product.objects.filter(category__name="Electronics")

Category.objects.filter(products__price__gte=1000)
```

---

10. Ordering and Limiting Results

Ascending

```
Product.objects.order_by("price")
```

Descending

```
Product.objects.order_by("-price")
```

Multiple fields

```
Product.objects.order_by("-created_at", "name")
```

Limit

```
Product.objects.all()[:10]
```

Offset and limit

```
Product.objects.all()[10:20]
```

Use deterministic ordering when paginating results.

---

11. `select_related()` and `prefetch_related()`

`select_related()`

```
products = Product.objects.select_related("category")

for product in products:
    print(product.category.name)
```

Best for ForeignKey and OneToOne relationships.

`prefetch_related()`

```
categories = Category.objects.prefetch_related("products")

for category in categories:
    for product in category.products.all():
        print(product.name)
```

Best for ManyToMany and reverse ForeignKey relationships.

Nested relationships

```
Product.objects.select_related("category__parent")
```

```
Category.objects.prefetch_related("products__tags")
```

Feature	`select_related()`	`prefetch_related()`
Mechanism	SQL JOIN	Separate queries
Typical use	ForeignKey/OneToOne	ManyToMany/reverse FK
Goal	Reduce N+1 queries	Reduce N+1 queries

---

12. Aggregation and Annotation

```
from django.db.models import Avg, Count, Max, Min, Sum
```

Aggregate

```
Product.objects.aggregate(
    total=Count("id"),
    average_price=Avg("price"),
    highest_price=Max("price"),
    lowest_price=Min("price"),
)
```

Annotation

```
Category.objects.annotate(
    product_count=Count("products")
)
```

```
for category in Category.objects.annotate(
    product_count=Count("products")
```

```
):
    print(category.name, category.product_count)
```

Conditional aggregation

```
from django.db.models import Count, Q

Category.objects.annotate(
    active_products=Count(
        "products",
        filter=Q(products__is_active=True),
    )
)
```

Watch for duplicate counts when joining multiple relationships.

---

13. F Expressions

```
from django.db.models import F
```

Increment stock

```
Product.objects.filter(pk=1).update(
    stock=F("stock") + 1
)
```

Increase prices

```
Product.objects.update(
    price=F("price") * 1.10
)
```

Compare fields

```
Product.objects.filter(
    stock__gt=F("reserved_stock")
)
```

`F()` expressions perform calculations using database field values and help avoid some read-modify-write race conditions.

---

14. Conditional Expressions

Case and When

```
from django.db.models import Case, When, Value, CharField

products = Product.objects.annotate(
    stock_status=Case(
        When(stock=0, then=Value("Out of stock")),
        When(stock__lt=5, then=Value("Low stock")),
        default=Value("Available"),
        output_field=CharField(),
    )
)
```

Conditional update

```
from django.db.models import Case, When, Value, IntegerField

Product.objects.update(
    discount_percent=Case(
        When(price__gte=5000, then=Value(10)),
        default=Value(0),
        output_field=IntegerField(),
    )
)
```

The example requires a `discount_percent` field on the model.

---

15. Subquery and Exists

Subquery

```
from django.db.models import OuterRef, Subquery

latest_product = (
    Product.objects
    .filter(category_id=OuterRef("pk"))
    .order_by("-created_at")
)

categories = Category.objects.annotate(
    latest_product_name=Subquery(
        latest_product.values("name")[:1]
    )
)
```

Exists

```
from django.db.models import Exists, OuterRef

active_products = Product.objects.filter(
    category_id=OuterRef("pk"),
    is_active=True,
)

categories = Category.objects.annotate(
    has_active_product=Exists(active_products)
)
```

`Exists()` is useful when you need to know whether related records exist.

---

16. Managers and Custom QuerySets

Custom manager

```
class ActiveProductManager(models.Manager):
    def get_queryset(self):
        return super().get_queryset().filter(is_active=True)
```

Custom QuerySet

```
class ProductQuerySet(models.QuerySet):
    def active(self):
        return self.filter(is_active=True)

    def in_stock(self):
        return self.filter(stock__gt=0)
```

Attach it to a model:

```
class Product(models.Model):
    name = models.CharField(max_length=150)
    stock = models.PositiveIntegerField(default=0)
    is_active = models.BooleanField(default=True)

    objects = ProductQuerySet.as_manager()
```

Usage:

```
Product.objects.active().in_stock()
```

---

17. Database Transactions

Atomic block

```
from django.db import transaction


@transaction.atomic
def create_product_and_log():
    product = Product.objects.create(
        name="Mouse",
        price="800.00",
        stock=10,
    )

    return product
```

Explicit transaction

```
with transaction.atomic():
    # Database operation 1
    # Database operation 2
    pass
```

Row locking

```
from django.db import transaction


with transaction.atomic():
    product = (
        Product.objects
        .select_for_update()
        .get(pk=1)
    )
```

```
    if product.stock > 0:
        product.stock -= 1
        product.save(update_fields=["stock"])
```

Locking behavior depends on the database backend. Keep transactions short and handle external side effects carefully.

---

18. Raw SQL

`raw()`

```
products = Product.objects.raw(
    """
    SELECT *
    FROM products_product
    WHERE price >= %s
    """,
    [1000],
)
```

Cursor

```
from django.db import connection


with connection.cursor() as cursor:
    cursor.execute(
        "SELECT id, name FROM products_product WHERE id = %s",
        [1],
    )
    rows = cursor.fetchall()
```

Always use parameterized SQL. Never concatenate untrusted user input into SQL strings.

---

19. Query Optimization and Indexes

Indexes

```
class Product(models.Model):
    name = models.CharField(max_length=150)
    price = models.DecimalField(max_digits=10, decimal_places=2)
    is_active = models.BooleanField(default=True)

    class Meta:
        indexes = [
            models.Index(fields=["price"]),
            models.Index(fields=["is_active", "price"]),
        ]
```

Execution plan

```
print(
    Product.objects.filter(price__gte=1000).explain()
)
```

Optimization checklist

- Identify slow queries.
- Reduce N+1 queries.
- Add indexes only when justified.
- Use pagination.
- Avoid loading unnecessary columns.
- Use bulk operations appropriately.
- Inspect execution plans.
- Benchmark with realistic data.

Indexes improve some reads but consume storage and can slow writes.

---

20. Bulk Operations

Bulk create

```
products = [
    Product(name="Mouse", price="800.00", stock=10),
    Product(name="Keyboard", price="1500.00", stock=20),
    Product(name="Monitor", price="12000.00", stock=5),
]

Product.objects.bulk_create(products, batch_size=500)
```

Bulk update

```
products = list(Product.objects.filter(is_active=True))

for product in products:
    product.stock += 5

Product.objects.bulk_update(
    products,
    ["stock"],
    batch_size=500,
)
```

Bulk delete

```
Product.objects.filter(stock=0).delete()
```

Bulk create/update generally do not call every instance's `save()` method. Do not rely on ordinary save signals or custom save logic for these operations.

---

21. Multiple Databases

Configuration

```
DATABASES = {
    "default": {
        "ENGINE": "django.db.backends.postgresql",
        "NAME": "main_db",
        "USER": "db_user",
        "PASSWORD": "load-from-environment",
        "HOST": "localhost",
        "PORT": "5432",
```

```
    },
    "analytics": {
        "ENGINE": "django.db.backends.postgresql",
        "NAME": "analytics_db",
        "USER": "db_user",
        "PASSWORD": "load-from-environment",
        "HOST": "localhost",
        "PORT": "5432",
    },
}
```

Query a database

```
Product.objects.using("default").all()
Product.objects.using("analytics").all()
```

Save to a database

```
product.save(using="default")
```

Use database routers when routing rules become complex. Ordinary Django transactions do not provide distributed atomicity across independent databases.

---

22. Fixtures

Export

```
python manage.py dumpdata products.Product --indent 2 > products.json
```

Import

```
python manage.py loaddata products.json
```

Use fixtures for sample data, reference data and reproducible development environments. Avoid committing sensitive production records.

---

23. Django Signals

`pre_save`

```
from django.db.models.signals import pre_save
from django.dispatch import receiver


@receiver(pre_save, sender=Product)
def normalize_product_name(sender, instance, **kwargs):
    instance.name = instance.name.strip()
```

`post_save`

```
from django.db.models.signals import post_save
from django.dispatch import receiver


@receiver(post_save, sender=Product)
def product_created(sender, instance, created, **kwargs):
```

```
    if created:
        print(f"New product: {instance.name}")
```

`m2m_changed`

Use this signal to respond to many-to-many relationship changes.
Signals can introduce hidden side effects. Prefer explicit service functions for important business workflows when appropriate.

---

24. Abstract Models and Proxy Models

Abstract model

```
class TimeStampedModel(models.Model):
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)

    class Meta:
        abstract = True


class Product(TimeStampedModel):
    name = models.CharField(max_length=150)
    price = models.DecimalField(max_digits=10, decimal_places=2)


class Category(TimeStampedModel):
    name = models.CharField(max_length=100)
```

An abstract model does not create its own database table.

Proxy model

```
class ProductByName(Product):
    class Meta:
        proxy = True
        ordering = ["name"]
```

A proxy model uses the existing model table.

---

25. Validation, Constraints and Security

Model validation

```
from django.core.exceptions import ValidationError


class Product(models.Model):
    name = models.CharField(max_length=150)
    price = models.DecimalField(max_digits=10, decimal_places=2)

    def clean(self):
        if self.price < 0:
            raise ValidationError({
                "price": "Price cannot be negative."
            })
```

Database constraint

```
class Product(models.Model):
    name = models.CharField(max_length=150)
    price = models.DecimalField(max_digits=10, decimal_places=2)
    stock = models.PositiveIntegerField(default=0)

    class Meta:
        constraints = [
            models.CheckConstraint(
                condition=models.Q(price__gte=0),
                name="product_price_nonnegative",
            ),
        ]
```

Check the constraint API for your Django version.

Validation layers

1. Form or serializer validation
2. Model validation
3. Database constraints
4. Authorization checks

`save()` does not automatically call `full_clean()`. Bulk operations may bypass model validation, so database integrity constraints are important.

---

26. Python and Django Introspection

Get model fields

```
from products.models import Product

for field in Product._meta.get_fields():
    print(field.name, field.__class__.__name__)
```

Get concrete fields

```
for field in Product._meta.concrete_fields:
    print(field.name, field.get_internal_type())
```

Model metadata

```
print(Product._meta.db_table)
print(Product._meta.label)
print(Product._meta.pk.name)
```

Inspect a field

```
field = Product._meta.get_field("price")

print(field.name)
print(field.null)
```

Introspection is useful for debugging, metadata tools, dynamic reporting and generic utilities.

---

27. Django ORM Interview Questions

Beginner

Q1. What is ORM?

A tool that maps Python objects to database records.

Q2. What does `filter()` return?

A QuerySet.

Q3. What exceptions can `get()` raise?

`DoesNotExist` and `MultipleObjectsReturned`.

Q4. What is the difference between `first()` and `get()`?

`first()` returns the first result or `None`; `get()` expects exactly one result.

Q5. What does `values()` do?

Returns dictionaries containing selected fields.

Intermediate

Q6. What is QuerySet laziness?

Query execution is deferred until results need to be evaluated.

Q7. What are Q objects?

Objects used to construct complex AND, OR and NOT conditions.

Q8. Why use F expressions?

To perform calculations using database field values.

Q9. What is the difference between `annotate()` and `aggregate()`?

`annotate()` adds calculated values to individual results; `aggregate()` returns summary values.

Q10. What is the difference between `select_related()` and `prefetch_related()`?

The first typically uses SQL joins for single-valued relations; the second uses separate queries for related collections.

Advanced

Q11. What is the N+1 query problem?

Unnecessary repeated queries for related records.

Q12. What does `transaction.atomic()` do?

Manages an atomic database transaction.

Q13. What does `select_for_update()` do?

Locks selected rows on supported databases while the transaction is active.

Q14. What is the limitation of `bulk_create()`?

It generally does not call each instance's `save()` method or ordinary save signals.

Q15. Why use `Subquery` and `Exists`?

To perform correlated database queries and existence checks.

Q16. How do you inspect generated SQL?

```
print(Product.objects.filter(price__gte=1000).query)
```

Q17. How do you inspect a query plan?

```
print(Product.objects.filter(price__gte=1000).explain())
```

Q18. When should you add a database index?

When the query workload and execution plan show that the index is beneficial.

---

28. Quick ORM Cheat Sheet

Task	Django ORM
All records	`Product.objects.all()`
Filter	`filter(price__gte=100)`
Exclude	`exclude(stock=0)`
Single record	`get(pk=1)`
First record	`first()`
Existence	`exists()`
Count	`count()`
Create	`create(name="Mouse", ...)`
Update instance	`obj.save()`
Bulk update	`update(stock=10)`
Delete	`delete()`
Ascending	`order_by("price")`
Descending	`order_by("-price")`
Select columns	`values("name",`

	`"price"` `)`
Tuple results	`values_list("id", flat=True)`
Distinct	`distinct()`
SQL JOIN	`select_related("category")`
Related prefetch	`prefetch_related("tags")`
Aggregate	`aggregate(Count("id"))`
Annotation	`annotate(total=Count("products"))`
OR condition	`filter(Q(a=1) | Q(b=2))`
Increment field	`update(stock=F("stock") + 1)`
Atomic block	`transaction.atomic()`
Execution plan	`explain()`
Bulk create	`bulk_create(objects)`

Bulk update	`bulk_update(objects, fields)`

---

29. Learning Roadmap

Week 1: Database and ORM Basics

- Database tables, keys and constraints
- Models and field types
- Migrations
- Create, read, update and delete

Week 2: QuerySets and Relationships

- `filter()`, `exclude()` and Q objects
- Field lookups and ordering
- ForeignKey, OneToOne and ManyToMany
- Related object queries

Week 3: Advanced ORM

- `select_related()` and `prefetch_related()`
- Aggregation and annotation
- F expressions and conditional expressions
- Subquery, Exists and indexes

Week 4: Production Patterns

- Transactions and row locking
- Custom managers and QuerySets
- Bulk operations and fixtures
- Tests and query optimization
- Capstone project

Daily practice routine

- 20 minutes: Study concepts
- 30 minutes: Write ORM queries
- 20 minutes: Solve problems
- 10 minutes: Inspect SQL and query counts

---

30. Capstone Project — Inventory Management ORM

Models

```
from django.conf import settings
from django.db import models


class Category(models.Model):
    name = models.CharField(max_length=100, unique=True)
```

```
class Product(models.Model):
    category = models.ForeignKey(
        Category,
        on_delete=models.PROTECT,
        related_name="products",
    )
    name = models.CharField(max_length=150)
    price = models.DecimalField(max_digits=10, decimal_places=2)
    stock = models.PositiveIntegerField(default=0)
    is_active = models.BooleanField(default=True)


class Order(models.Model):
    customer = models.ForeignKey(
        settings.AUTH_USER_MODEL,
        on_delete=models.PROTECT,
        related_name="orders",
    )
    created_at = models.DateTimeField(auto_now_add=True)


class OrderItem(models.Model):
    order = models.ForeignKey(
        Order,
        on_delete=models.CASCADE,
        related_name="items",
    )
    product = models.ForeignKey(
        Product,
        on_delete=models.PROTECT,
    )
    quantity = models.PositiveIntegerField()
    unit_price = models.DecimalField(
        max_digits=10,
        decimal_places=2,
    )
```

Practice tasks

1. Retrieve all active products.
2. Retrieve products priced above 1,000.
3. Count products in each category.
4. Calculate the total amount for each order.
5. Find categories without active products.
6. Optimize product listing with `select_related()`.
7. Optimize order listing with `prefetch_related()`.
8. Increment stock using `F()`.
9. Use transactions for order creation.
10. Inspect and reduce query counts.
11. Write tests for insufficient stock and concurrent updates.
12. Create a paginated REST API.

Sample solutions

Task 1: Active products

```
Product.objects.filter(is_active=True)
```

Task 2: Products above 1,000

```
Product.objects.filter(price__gt=1000)
```

Task 3: Product count per category

```
from django.db.models import Count

Category.objects.annotate(
    product_count=Count("products")
)
```

Task 4: Order totals

```
from django.db.models import F, Sum

Order.objects.annotate(
    total_amount=Sum(
        F("items__quantity") * F("items__unit_price")
    )
)
```

Task 5: Categories without active products

```
from django.db.models import Exists, OuterRef

active_products = Product.objects.filter(
    category_id=OuterRef("pk"),
    is_active=True,
)

Category.objects.filter(~Exists(active_products))
```

Task 6: Optimized product listing

```
Product.objects.select_related("category")
```

Task 7: Optimized order listing

```
Order.objects.prefetch_related("items__product")
```

Task 8: Atomic stock decrement

```
from django.db.models import F

updated = Product.objects.filter(
    pk=product_id,
    stock__gte=quantity,
).update(stock=F("stock") - quantity)

if updated == 0:
    raise ValueError("Product unavailable or insufficient stock")
```

For production order processing, perform stock changes and order creation within an appropriate transaction, and handle failures and concurrent requests correctly.

Final Study Strategy

Follow this order:

1. Django fundamentals: URLs, views, templates, forms and models.
2. ORM mastery: CRUD, QuerySets, relationships and annotations.
3. Backend engineering: DRF, authentication, permissions and tests.
4. Advanced ORM: Transactions, indexes, subqueries and optimization.
5. Production: PostgreSQL, caching, Celery, Docker and CI/CD.
6. Portfolio: Build, test, document and deploy a complete API.

Core principle: Do not just memorize ORM syntax. Learn what each query returns, how many SQL queries it executes, which database constraints apply, and what happens under concurrent requests.